Quantized DeiT-Tiny Acceleration on RV64IMC

This repository contains an ISA-level implementation and evaluation of W8A8 quantized DeiT-Tiny kernels on bare-metal RV64IMC.

The current midterm-stage implementation adds two custom instructions to Spike:

- `Xqmac8`: packed 8-way signed INT8 multiply-accumulate
- `Xrequant`: Q31 fixed-point requantization with rounding and INT8 saturation

The project currently includes operator-level kernels and one complete connected DeiT-Tiny Transformer block.

---

1. Current Status

Implemented:

- [x] DeiT-Tiny workload inspection
- [x] W8A8 symmetric quantization
- [x] per-output-channel INT8 weight quantization
- [x] INT32 accumulation
- [x] fixed-point Q31 requantization
- [x] bare-metal RV64IMC runtime
- [x] `Xqmac8`
- [x] `Xrequant`
- [x] Spike custom instruction support
- [x] directed ISA tests
- [x] randomized ISA tests
- [x] FC1 / FC2
- [x] QKV
- [x] QK^T
- [x] integer Softmax
- [x] Attention x V
- [x] output projection
- [x] integer LayerNorm
- [x] GELU LUT
- [x] residual scale alignment
- [x] complete connected Transformer Block0
- [x] Block0 B0/X2 dynamic instruction profiling

Deferred to the later project stage:

- [ ] CV-Wally RTL integration
- [ ] cycle-accurate performance evaluation
- [ ] CPI analysis
- [ ] synthesis area

- [ ] Fmax / critical path
- [ ] full 12-block DeiT-Tiny inference
- [ ] ImageNet classification accuracy

---

2. Main Result

Complete connected Transformer Block0:

```
Useful MACs: 102,049,152
```

Dynamic instruction count:

Metric	B0 RV64IMC	X2
Program instructions	743,784,995	105,759,995
Main instructions	740,886,472	102,863,474
Instructions / MAC	7.260094	1.007980
MAC / instruction	0.137739	0.992083
Loads / MAC	2.018695	0.266749

Main workload dynamic-instruction reduction:

```
86.12%
```

B0/X2 main instruction-count ratio:

```
7.20x
```

Total program instruction-count ratio:

```
7.03x
```

These are ISA-level dynamic instruction results measured using Spike. They are **not** cycle-accurate hardware speedups.

---

3. Functional Result

The complete Block0 output contains:

```
197 x 192 = 37,824 INT8 values
```

The Spike X2 result matches the Python integer reference bit-exactly.

```
Block0 X2 verify = 0
```

No unintended floating-point instructions were found in the final bare-metal Block0 executable.

Quantization quality relative to the FP32 Block0 reference:

```
Final MAE     : 0.078367
Final Max Err : 0.438896
Final SQNR    : 19.63 dB
```

---

4. Repository Structure

```
.
├── baremetal/
│   ├── crt0.S
│   ├── link.ld
│   └── smoke.c
│
├── model/
│   ├── check_deit.py
│   ├── inspect_deit.py
│   ├── dump_block0.py
│   ├── dump_block0_mlp.py
│   ├── dump_fc1.py
│   ├── dump_fc2.py
│   └── dump_qkv.py
│
├── reference/
│   ├── build_av0.py
│   ├── build_block0_connected.py
│   ├── build_mlp0_w8a8.py
│   ├── build_proj0.py
│   ├── build_qkt.py
│   ├── build_softmax0.py
│   ├── quantize_fc1.py
│   ├── quantize_fc2.py
│   └── quantize_qkv.py
│
├── scripts/
│   ├── export_*_c.py
│   ├── profile_block0_b0.py
```

```
│   ├── profile_block0_x2.py
│   └── profile_*.py
│
├── src/
│   ├── generated/
│   ├── block0_bench.c
│   ├── block0_x2_verify.c
│   ├── fc1_*.c
│   ├── fc2_*.c
│   ├── qkv_*.c
│   ├── qkt_*.c
│   ├── av0_*.c
│   ├── proj0_*.c
│   ├── mlp0_*.c
│   └── softmax0_*.c
│
├── tests/
│   ├── dot8_scalar_bench.c
│   ├── dot8_scalar_lb_bench.c
│   ├── dot8_xqmac8_bench.c
│   ├── isa_random_test.c
│   ├── xqmac8_test.c
│   └── xrequant_test.c
│
├── results/
│   └── *_profile.txt
│
├── patches/
│   └── spike-xqmac-xrequant.patch
│
├── DESIGN.md
├── EXPERIMENTS.md
└── README.md
```

---

5. Software Environment

The project has been developed using:

```
Host             : Windows + WSL2 Ubuntu
RISC-V ISA       : RV64IMC
RISC-V GCC       : 13.2.0
GNU binutils     : 2.42
Python           : 3.x
NumPy            : 2.5.4
PyTorch          : 2.14.1+cpu
timm             : 1.0.30
```

Spike base commit:

```
0bff121
```

The custom Spike patch is:

```
patches/spike-xqmac-xrequant.patch
```

---

6. Python Environment

Create a Python virtual environment if needed.

On Windows PowerShell:

```
python -m venv .venv
.venv\Scripts\Activate.ps1

pip install numpy torch timm
```

The exact package versions used for the current experiments are listed in the previous section.

---

7. Build the Python References

The current workflow first dumps FP32 data from DeiT-Tiny and then constructs integer references.

Representative commands:

```
python model\dump_block0.py
python model\dump_block0_mlp.py
python model\dump_qkv.py
python model\dump_fc1.py
python model\dump_fc2.py
```

Build quantized operator references:

```
python reference\quantize_qkv.py
python reference\build_qkt.py
python reference\build_softmax0.py
python reference\build_av0.py
python reference\build_proj0.py
python reference\build_block0_connected.py
```

Export the connected Block0 data to C:

```
python scripts\export_block0_connected_c.py
```

This generates:

```
src/generated/block0_data.h
src/generated/block0_data.c
```

---

8. Build Spike with Custom Instructions

Clone or prepare the Spike source tree at the base commit:

```
0bff121
```

Apply the project patch:

```
git checkout 0bff121

git apply \
    /path/to/deit-rv64imc-isa/patches/spike-xqmac-xrequant.patch
```

The patch modifies:

```
riscv/encoding.h
riscv/riscv.mk.in
riscv/insns/xqmac8.h
riscv/insns/xrequant.h
```

The patch can be checked before applying:

```
git apply --check \
    /path/to/deit-rv64imc-isa/patches/spike-xqmac-xrequant.patch
```

After applying the patch, rebuild Spike using the local Spike build configuration.

The development environment installs Spike under:

```
$HOME/riscv/install/bin/spike
```

9. Bare-Metal Build Flags

The Block0 experiments use:

```
COMMON_FLAGS="\
-march=rv64imc \
-mabi=lp64 \
-mcmodel=medany \
-msmall-data-limit=0 \
-O2 \
-ffreestanding \
-fno-builtin \
-nostdlib \
-nostartfiles \
-Isrc \
-Wl,-T,baremetal/link.ld"
```

10. Verify Complete Block0

Build the baseline verification binary:

```
riscv64-unknown-elf-gcc \
    $COMMON_FLAGS \
    -DUSE_X2=0 \
    -DVERIFY=1 \
    baremetal/crt0.S \
    src/block0_bench.c \
    src/generated/block0_data.c \
    -o build/block0_b0_check.elf
```

Run:

```
spike \
    --isa=rv64imc \
    build/block0_b0_check.elf

echo "Block0 B0 check = $?"
```

Expected result:

```
Block0 B0 check = 0
```

Build X2 verification:

```
riscv64-unknown-elf-gcc \
    $COMMON_FLAGS \
    -DUSE_X2=1 \
    -DVERIFY=1 \
    baremetal/crt0.S \
    src/block0_bench.c \
    src/generated/block0_data.c \
    -o build/block0_x2_check.elf
```

Run:

```
spike \
    --isa=rv64imc \
    build/block0_x2_check.elf

echo "Block0 X2 check = $?"
```

Expected result:

```
Block0 X2 check = 0
```

---

11. Build Block0 Benchmarks

Baseline:

```
riscv64-unknown-elf-gcc \
    $COMMON_FLAGS \
    -DUSE_X2=0 \
    -DVERIFY=0 \
    baremetal/crt0.S \
    src/block0_bench.c \
    src/generated/block0_data.c \
    -o build/block0_b0_bench.elf
```

X2:

```
riscv64-unknown-elf-gcc \
    $COMMON_FLAGS \
    -DUSE_X2=1 \
    -DVERIFY=0 \
    baremetal/crt0.S \
    src/block0_bench.c \
```

```
    src/generated/block0_data.c \
    -o build/block0_x2_bench.elf
```

Run both:

```
spike --isa=rv64imc build/block0_b0_bench.elf
echo "B0 bench = $?"

spike --isa=rv64imc build/block0_x2_bench.elf
echo "X2 bench = $?"
```

Expected:

```
B0 bench = 0
X2 bench = 0
```

---

12. Generate Spike PC Histograms

Baseline:

```
spike \
    -g \
    --isa=rv64imc \
    build/block0_b0_bench.elf \
    2> results/block0_b0_pc_hist.txt
```

X2:

```
spike \
    -g \
    --isa=rv64imc \
    build/block0_x2_bench.elf \
    2> results/block0_x2_pc_hist.txt
```

---

13. Generate Disassembly

```
riscv64-unknown-elf-objdump \
    -d build/block0_b0_bench.elf \
    > results/block0_b0_disasm.txt
```

```
riscv64-unknown-elf-objdump \
    -d build/block0_x2_bench.elf \
    > results/block0_x2_disasm.txt
```

---

14. Generate Dynamic Instruction Profiles

```
python3 scripts/profile_block0_b0.py \
    | tee results/block0_b0_profile.txt

python3 scripts/profile_block0_x2.py \
    | tee results/block0_x2_profile.txt
```

Expected key results:

```
B0 main instructions:
740,886,472

X2 main instructions:
102,863,474
```

---

15. Check Code Size

```
riscv64-unknown-elf-size \
    -A build/block0_b0_bench.elf \
    | grep '^.text'

riscv64-unknown-elf-size \
    -A build/block0_x2_bench.elf \
    | grep '^.text'
```

Current result:

```
B0 .text : 2748 bytes
X2 .text : 2164 bytes
```

---

16. Check for Floating-Point Instructions

Generate disassembly:

```
riscv64-unknown-elf-objdump \
    -d build/block0_x2_check.elf \
    > results/block0_x2_disasm.txt
```

Search:

```
grep -E \
'\b(fadd|fsub|fmul|fdiv|fsqrt|flw|fld|fsw|fsd)\b' \
results/block0_x2_disasm.txt
```

Expected result:

```
no output
```

---

17. ISA Tests

Directed tests:

```
tests/xqmac8_test.c
tests/xrequant_test.c
```

Randomized test:

```
tests/isa_random_test.c
```

The randomized test currently executes:

```
10,000 Xqmac8 cases
10,000 Xrequant cases
```

with a fixed random seed.

---

18. Custom Instruction Encodings

Xqmac8

```
opcode = CUSTOM-0 = 0x0B
funct3 = 000
funct7 = 0000000

MATCH_XQMAC8 = 0x0000000b
MASK_XQMAC8  = 0xfe00707f
```

Xrequant

```
opcode = CUSTOM-0 = 0x0B
funct3 = 001
funct7 = 0000000

MATCH_XREQUANT = 0x0000100b
MASK_XREQUANT  = 0xfe00707f
```

See:

```
DESIGN.md
```

for detailed semantics.

---

19. Experiment Documentation

Detailed experiment results:

```
EXPERIMENTS.md
```

ISA and quantization design:

```
DESIGN.md
```

Important profile outputs:

```
results/dot8_fair_profile.txt
results/fc2_b0_profile.txt
results/fc2_b1_profile.txt
results/fc2_x1_profile.txt
results/fc2_x2_profile.txt
results/qkv_b0_profile.txt
```

```
results/qkv_x2_profile.txt
results/qkt_b0_profile.txt
results/qkt_x1_profile.txt
results/block0_b0_profile.txt
results/block0_x2_profile.txt
```

---

20. Git Checkpoints

Complete Block0 experiment checkpoint:

```
commit:
5ebd201

tag:
midterm-block0-v1
```

Spike patch commit:

```
f507d92
```

---

21. Current Architecture Conclusions

The current measurement-driven optimization path is:

```
RV64IMC baseline
      |
      v
scalar MAC bottleneck
      |
      v
Xqmac8
      |
      v
requantization overhead
      |
      v
Xrequant
      |
      v
complete X2 Block0
      |
      v
remaining bottlenecks:
load
address generation
```

```
loop control
RV64 sign extension
```

The complete Block0 X2 implementation reduces main dynamic instructions by:

```
86.12%
```

relative to B0.

---

22. Next Stage

The next project stage will focus on CV-Wally RTL integration.

Planned measurements include:

- kernel cycle count
- CPI
- pipeline stalls
- critical path
- Fmax
- synthesized area
- performance / area
- hardware cost of `Xqmac8`
- hardware cost of `Xrequant`

The main question is whether the large ISA-level dynamic-instruction reduction can be translated into proportional cycle-level gains without excessive hardware cost.